Tema 1.2:

Ingeniería de datos textuales

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo
© Prof. David Candal Ventureira - 2026

Tabla de contenidos

01 Adquisición

02 Limpieza

03 Gestión del corpus

04 Resumen

01

Adquisición

Adquisición: Estrategias de obtención de datos

1. **APIs comerciales/privadas**: La plataforma nos entrega los datos estructurados (JSON, XML).
   - Ventajas: Documentación clara, datos limpios, legalidad explícita.
   - Desventajas: Pueden ser de pago y tener restricciones de acceso.
   - Ejemplos: Twitter (X), Reddit, The New York Times API.
2. **Web scraping**: Extraer información del código fuente de una WEB (HTML).
   - Retos: Bloqueos anti-bots, cambios en la estructura de la web, limpieza de ruido (anuncios, menús…).
3. **Documentos**: Extracción de texto desde archivos ofimáticos o digitalizados.
   - Formatos: PDF, DOCX, PPTX, TXT.
   - Retos: Extracción del texto, procesamiento de tablas, reconocimiento de texto en imágenes.

Adquisición: APIs y servicios WEB

Conceptos clave:

- **Endpoints**: URLs específicas para pedir recursos (ej: /2/tweets/search/recent?query=…).
- **Autenticación**: Mecanismos para identificar nuestra aplicación.
  - API Key / Secret: Usuario y contraseña de la API.
  - Bearer token / OAuth: Permisos temporales o delegados.
- **Cuotas y rate limits**: Límite de peticiones por minuto/mes (ej: "429 Too Many Requests"). Necesidad de implementar estrategias de backoff (espera y reintento).

Clientes:

- Clientes genéricos con soporte REST: wget, curl, requests, Aiohttp (Python).
- Librerías específicas de la API: Simplifican el acceso a los datos de la API.

Adquisición: WEB scraping I

Estrategias de WEB scraping

- **APIs de carga de datos (XHR / Fetch):**
  - Aprovechar las APIs utilizadas por las WEBs para cargar dinámicamente datos.
  - Datos estructurados (JSON, XML…), ligero y sencillo (no necesitamos cargar el resto de recursos de la WEB).
  - Opción generalmente no disponible para obtener datos textuales.
  - Herramientas: Requests, wget, aiohttp…
- **WEBs estáticas (HTML) :**
  - Extraer los textos relevantes del HTML de la WEB.
  - Datos estructurados (HTML) pero sucios (boilerplate, anuncios…).
  - Opción mayoritariamente disponible.
- **WEBs dinámicas (HTML + Javascript):**
  - Los datos (visibles para el usuario) se adquieren, modifican o eliminan dinámicamente (discrepancia entre DOM y HTML). Ejemplo: protección anti-bots.
  - Solución: headless browsers (simulan un usuario navegando por la WEB).
  - Problema: 25-50x coste de CPU y RAM.

Adquisición: WEB scraping II

Adquisición de datos textuales de WEBs estáticas:

- **Cliente:**
  - Requests (Python): Sencillo, bloqueante, anticuado.
  - Aiohttp (Python): Moderno, asíncrono. Preparado para descarga masiva de WEBs.
- **Limitaciones:**
  - Estas técnicas no comprueban las modificaciones realizadas por Javascript, sólo extraen datos del HTML original.

Adquisición: WEB scraping III

HTML vs DOM:

- **HTML:**
  - Código fuente original: Estructura, datos de partida (texto, imágenes), estética (CSS) y scripts Javascript.
  - Estático, inmutable.
  - Herramientas como aiohttp y requests sólo pueden obtener texto presente en este fichero.
  - Accesible desde el navegador mediante Ctrl + U.
- **DOM:**
  - Resultado de aplicar las modificaciones de Javascript al HTML original.
  - Dinámico: Puede cambiar periódicamente o tras acciones (scroll, click, etc).
  - Accesible desde el navegador mediante F12.

Adquisición: WEB scraping IV

HTML vs DOM:

- Ejemplo: https://d-candal.github.io/test-web/
- Observa las diferencias entre el HTML original (CTRL+U) y el DOM (F12).

Adquisición: WEB scraping V

Adquisición de datos textuales de WEBs dinámicas:

- Headless browsers:
  - Navegadores web (Chrome, Firefox) controlados por código.
  - Permiten renderizar el DOM completo (HTML + modificaciones JS) e interactuar con la página.
- Herramientas:
  - Selenium: Estándar para WEB scraping de WEBs dinámicas. Gran ecosistema.
  - Playwright: Alternativa moderna. Más optimizado. Incluye funcionalidades de serie (ej: esperar a que los elementos estén listos…).
- Funcionalidades:
  - Operaciones sobre el DOM (HTML + operaciones JS).
  - Interacciones: scroll infinito, clicks, etc.
- Coste: Ejecutar un headless browser tiene un impacto significativamente superior a un sistema de consultas HTTP (aprox. 20-50x mayor carga).

Adquisición: WEB scraping VI

¿Sobre qué contenidos podemos hacer WEB scrapping?

1. **Licencia explícita:**
   - Contrato vinculante con el usuario sobre derechos de autor (uso del contenido), sus indicaciones prevalecen sobre otros mecanismos.
   - Creative Commons (CC-BY, CC0), Open Data…
2. **Términos de servicio (ToS):**
   - Contrato vinculante con el usuario sobre derechos de acceso al servicio.
   - En caso de no incluir licencia, los ToS prevalecen sobre otros mecanismos.
   - Ej: Prohibido minado de datos, prohíbo scraping, prohibido acceso automático…
3. **Protocolo robots.txt:**
   - Protocolo de cortesía que deben respetar los scripts automatizados a la hora de acceder a un dominio.
   - Disponible en: [dominio.TLD]/robots.txt.
   - No define derechos de uso, copyright…

Adquisición: WEB scraping VII

El protocolo robots.txt

1. **User-Agent:**
   - Indica a qué agentes aplican las reglas que se indican a continuación.
     - *: Cualquiera.
     - ChatGPT-User, wget…: User-agent de servicios/herramientas específicos.
     - El “robot” debe ceñirse a la entrada más específica.
2. **Allow/Disallow:**
   - Indican las rutas del dominio que el script tiene permitido o denegado acceder de manera automatizada.
3. **Sitemaps:**
   - Mapa de cortesía de los recursos relevantes accesibles a través del dominio.
4. **Crawl-delay:**
   - Segundos de espera que deben transcurrir entre peticiones (por debajo de este valor, la página puede marcar las consultas como DDoS).
5. **License:**
   - Enlace a la licencia de uso del dominio.

Adquisición: WEB scraping VIII

El protocolo robots.txt

```
# version: 945f36196057348d62861344edc121a150851e7f

# HTTPS www.url.tld

User-agent: *
Crawl-delay: 60
Sitemap: https://www.bbc.com/sitemaps/https-index-com-archive.xml
Sitemap: https://www.bbc.com/sitemaps/https-index-com-news.xml
...
Disallow: /asset/
Disallow: /bitesize/search$
Disallow: /bitesize/search/
...

User-agent: Amazonbot
Disallow: /

User-agent: Claude-Web
Disallow: /

User-agent: ClaudeBot
Disallow: /

User-agent: anthropic-ai
Disallow: /

User-agent: wget
Disallow: /
```

Adquisición: WEB scraping IX

Ejemplos de términos de uso y robots.txt:

- *BBC*:
  - https://www.bbc.com/robots.txt
  - https://www.bbc.com/pages/terms-of-use (art 7a)
- *La voz de Galicia*:
  - https://www.lavozdegalicia.es/robots.txt (wget...)
  - https://www.lavozdegalicia.es/docs/aviso_legal.htm (art 3)

Adquisición: Documentos I

Tipos de documentos:

- **Documentos textuales simples:**
  - JSON, YAML, CSV, TXT…
  - Texto plano o semi-estructurado.
  - Extraer el texto plano relevante es simple (ambigüedad mínima).
- **Documentos textuales complejos:**
  - XLSX: Tabular, similar a CSV. Contiene relaciones y contexto adicionales al texto, requiere algoritmos capaces de combinar ambos adecuadamente.
  - DOCX: Estructurado por secciones, párrafos… Similar a HTML. El orden del texto claro en el fichero puede diferir del visual (el correcto).
  - PPTX: Vectorial. Elementos (bloques de texto, imágenes…) colocados en coordenadas específicas de la página. El texto claro en el fichero puede no estar ordenado.
  - PDF: Similar a PPTX. Elementos posicionados en el documento, no necesariamente en el mismo orden que el visual.

Adquisición: Documentos II

Documentos PDF:

- Un PDF puede ser:
  - Nativo: Incluye texto claro con las coordenadas en las que se ubica.
  - Rasterizado: El texto en realidad se representa como una imagen (píxeles).
- La estructura interna depende de cómo se haya generado:
  - Word (monocolumna): Listado de párrafos, bloques de texto plano ordenados. Texto claro extraíble.
  - Powerpoint: Elementos ubicados por coordenadas (no necesariamente ordenados según su presentación). Texto claro extraíble.
  - Escaneado: Imagen colocada sobre la página. Es necesario convertir la imagen a texto.
- ¿Cómo distinguir un PDF nativo de uno raster?
  - Tratar de extraer texto del documento. Si no es coherente -> OCR.

Adquisición: Documentos III

Documentos PDF nativos:

- Herramientas:
  - PyMuPDF: Procesado rápido, puede fallar al agrupar textos.
  - pdfplumber: Procesado lento, agrupaciones de textos más precisas.
  - Resultado de las herramientas:
    - Texto.
    - Bounding boxes (coordenadas de la caja del bloque de texto).
    - Tamaño de fuente y tipo de letra.
- Retos:
  - Una tabla en un PDF es una combinación de líneas (ubicadas formando una tabla) y palabras/frases ubicadas en la posición de cada celda.
  - Orden de extracción de los textos: Dado que cada bloque textual es independiente, es posible que los textos no se extraigan en el orden adecuado.

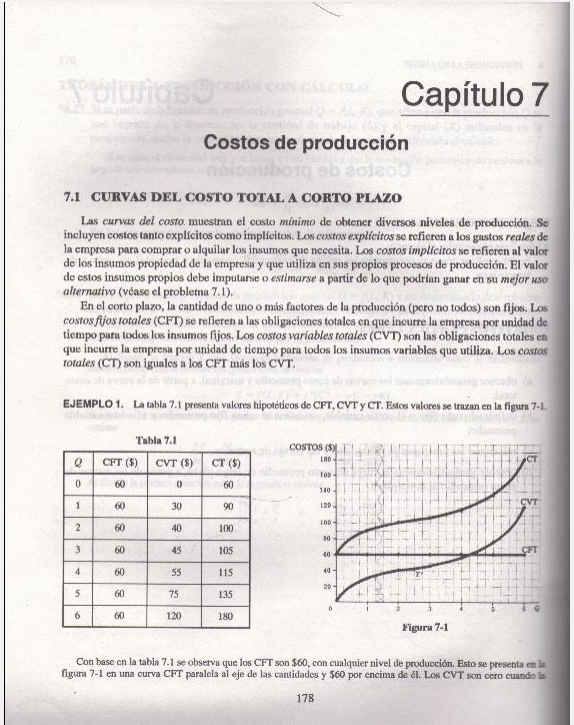
Adquisición: Documentos IV

Documentos PDF rasterizado (escaneado):

- Extraer el texto de la imagen incrustada en el PDF.
- Imágenes en mal estado -> Errores de extracción -> Datos ruidosos.
- Solución: preprocesado con visión por ordenador (OpenCV):
    - **Binarización**: Algoritmo de Otsu (valor umbral) para separar la imagen del texto del fondo del documento (diferencia de blancos).
    - **Enderezado**: Corrección de la inclinación del texto en la imagen, tratando de alinear las líneas.
    - **Denoising**: Eliminación de ruido salt-pepper mediante filtros.
    - **Aumento de resolución**: Reescalado a mayor DPI para maximizar la tasa de acierto.
- Tras ello, se aplica reconocimiento óptico de caracteres (OCR).
    - Herramientas: Tesseract. Devuelve texto y bounding boxes.

Adquisición: Documentos V

Ejemplo de PDF rasterizado:



Capítulo 7

Costos de producción

7.1 CURVAS DEL COSTO TOTAL A CORTO PLAZO

Las *curvas del costo* muestran el costo *mínimo* de obtener diversos niveles de producción. Se incluyen costos tanto explícitos como implícitos. Los *costos explícitos* se refieren a los gastos *reales* de la empresa para comprar o alquilar los insumos que necesita. Los *costos implícitos* se refieren al valor de los insumos propiedad de la empresa y que utiliza en sus propios procesos de producción. El valor de estos insumos propios debe imputarse o *estimarse* a partir de lo que podrían ganar en su *mejor uso alternativo* (véase el problema 7.1).

En el corto plazo, la cantidad de uno o más factores de la producción (pero no todos) son fijos. Los *costos fijos totales* (CFT) se refieren a las obligaciones totales en que incurre la empresa por unidad de tiempo para todos los insumos fijos. Los *costos variables totales* (CVT) son las obligaciones totales en que incurre la empresa por unidad de tiempo para todos los insumos variables que utiliza. Los *costos totales* (CT) son iguales a los CFT más los CVT.

EJEMPLO 1. La tabla 7.1 presenta valores hipotéticos de CFT, CVT y CT. Estos valores se trazan en la figura 7-1.

Tabla 7.1

Q	CFT ($)	CVT ($)	CT ($)
0	60	0	60
1	60	30	90
2	60	40	100
3	60	45	105
4	60	55	115
5	60	75	135
6	60	120	180

Figura 7-1

Con base en la tabla 7.1 se observa que los CFT son $60, con cualquier nivel de producción. Esto se presenta en la figura 7-1 en una curva CFT paralela al eje de las cantidades y $60 por encima de él. Los CVT son cero cuando la

178

Adquisición: Documentos VI

Estructura de los textos en PDF:

- PyMuPDF, pdfplumber y Tesseract nos permiten extraer el texto y su ubicación en la página de un PDF, pero no nos proporcionan qué función tiene el texto en el documento (título, párrafo, pie de página…).
- Análisis de la disposición del documento (document layout analysis, DLA):
    - Análisis geométrico del documento (posición y tamaño de los bloques de texto) para determinar su tipo.
    - Herramientas:
        - LayoutLM: Toma el texto y las bounding boxes para etiquetar cada bloque de texto.
        - YOLO: Analiza visualmente el documento para obtener y etiquetar bounding boxes. Las bounding boxes se comparan con las obtenidas en la extracción de texto para etiquetar cada bloque.
        - Unstructured: Extracción de texto (nativo o rasterizado) + DLA.

02

Limpieza

Limpieza: Reparación de codificación I

¿Cómo se codifican los textos?

- Los textos se almacenan y cargan de disco en bytes.
- Existen diferentes formatos de codificación.

Texto: café-pingüino

Codificación	Representación (Hex)	Explicación
UTF-8 (NFC)	63 61 66 C3 A9 2D 70 69 6E 67 C3 BC 69 6E 6F c a f é - p i n g ü i n o	é y ü ocupan 2 bytes.
ASCII	63 61 66 ?? 2D 70 69 6E 67 ?? 69 6E 6F c a f é - p i n g ü i n o	ASCII solo admite letras inglesas, no puede representar é. (?? -> 3F)
Windows-1252	63 61 66 E9 2D 70 69 6E 67 FC 69 6E 6F c a f é - p i n g ü i n o	

Limpieza: Reparación de codificación II

¿Qué pasa cuando leemos un fichero? (Python)

- Si no indicamos el formato de codificación al abrir el fichero (encoding="utf-8“), Python usa la codificación predeterminada del sistema (cp1252, UTF-8…).
- Si el fichero encuentra un carácter ilegal (bytes que no se pueden convertir en un carácter para la codificación seleccionada), se produce un fallo al leer el fichero (salvo si se indica errors="replace“ para sustituir los caracteres por).
- Tras decodificar el texto, se almacena en un string como una lista de caracteres Unicode:
    - Dos textos iguales provenientes de dos codificaciones diferentes son iguales porque sus strings tienen la misma representación Unicode, pero…. ->

Limpieza: Reparación de codificación III

… dos strings que PARECEN iguales pueden tener diferentes representaciones:

- Los strings pueden contener caracteres invisibles:
  - Zero-width space (ZWSP): Separación invisible entre palabras.
  - Zero-width joiner (ZWJ): Permite generar caracteres complejos en base a dos caracteres individuales (lenguajes hindi, nepalí… y emojis).
  - Non-breaking space (NBSP): Espacio que no permite la separación de las dos palabras en múltiples líneas.
- Adicionalmente, existen dos representaciones UTF-8:

Texto: café		
UTF-8	Representación (Unicode)	Representación (Hex)
NFC	U+0063 (c) U+0061 (a) U+0066 (f) U+00E9 (é)	63 61 66 C3 A9
NFD	U+0063 (c) U+0061 (a) U+0066 (f) U+0065 (e) U+0301 (◌́)	63 61 66 65 CC 81

Limpieza: Reparación de codificación IV

Ejemplo

```python
from unicodedata import normalize as unc_normalize

# Errores de decodificación
texto = "café mañana"
print(f"UTF-8: {texto.encode('utf-8').decode('utf-8', errors='replace')}")
print(f"ASCII: {texto.encode('utf-8').decode('ascii', errors='replace')}")
print(f"Windows-1252: {texto.encode('cp1252').decode('utf-16',
errors='replace')}")

# Caracteres invisibles
texto2 = "café\u00A0ma\u200Dña\u200Bna"
print(f"\nTexto original: '{texto}'. Texto modificado: '{texto2}'.
¿Iguales?: {texto == texto2}")

# Comprobación de textos iguales en diferentes formatos
texto_1252 = texto.encode('cp1252').decode('cp1252')
texto_utf8 = texto.encode('utf-8').decode('utf-8')
print(f"\n¿Iguales?: {texto_1252 == texto_utf8}")

# Errores de normalización UTF-8
texto_nfc = unc_normalize('NFC', texto)
texto_nfd = unc_normalize('NFD', texto)
print(f"\nTexto en NFC: '{texto_nfc}'. Texto en NFD: '{texto_nfd}'.
Iguales? {texto_nfc == texto_nfd}")
```

```
UTF-8: café mañana
ASCII: caf◆◆ ma◆◆ana
Windows-1252: 慣◆洚◆渭◆

Texto original: 'café mañana'. Texto modificado: 'café mañana'.
 ¿Iguales?: False

¿Iguales?: True

Texto en NFC: 'café mañana'. Texto en NFD: 'café mañana'. Igual
es? False
```

Limpieza: Reparación de codificación V

¿Cómo solucionamos estos problemas? (Python)

- Reparación de problemas de codificación:
  - Fix text for you (ftfy, Python):
    - Conversión a latin-1 (fuerza que sea codificable, aunque generando mojibakes).
    - Limpieza con la librería.
    - Ejemplo:

```
t1 = ftfy.fix_text(texto.encode('utf-8').decode('latin-1'))
t2 = ftfy.fix_text(texto.encode('cp1252').decode('latin-1'))
print(f"\nSon los 3 textos iguales? {texto == t1 == t2}")  # True
```

- Los caracteres invisibles requieren ser eliminados de manera independiente.
- Los problemas de normalización UTF-8 se corrigen normalizando el texto a NFC o NFD.

Limpieza: Reducción de ruido I

Limpieza de ruido en documentos:

- Extracción del texto + DLA y filtrado de cabeceras, pies de página, etc.

Limpieza de ruido en HTMLs (boilerplate):

- Uso de expresiones regulares para patrones de texto irrelevante (ej: Suscríbete, Leer más, Comentarios…).
- Reconocimiento de la estructura del fichero HTML mediante parser:
  - BeautifulSoup (Python): Simple, pero lento. Mayor documentación.
  - Selectolax (Python): Hasta 20x más rápido que BeautifulSoup.
- Análisis de las etiquetas estructurales, uso de patrones y análisis de densidad de texto para extraer elementos relevantes.
- Trafilatura: Herramienta sencilla de WEB scrapping para extracción rápida y heurística de texto (no diferencia títulos del cuerpo de los artículos).

Limpieza: Reducción de ruido II

Selectores (parsing):

- Permiten extraer elementos concretos del HTML en función de su estructura.
- Clave para identificar el texto relevante dentro de grandes volúmenes de páginas web.

- **XPATH:**
  - Basado en grafos. Lento.
  - Permite navegación compleja (transitar a nodos ascendentes y descendientes).
  - Ej:
    - //article//div[@class='content']/p
- **Selectores CSS:**
  - Búsqueda lineal por patrones (análogo a CTRL + F). Rápido.
  - Permite consultar nodos hijos, pero no permite subir en la jerarquía.
  - Ej:
    - article div.content > p

- Recomendación: Buscar elementos relevantes en base a etiquetas semánticas (id o name) o estructurales (article, main, section…).
- Herramientas modernas permiten búsquedas complejas (ej: CSS, rápido + XPATH, por contenido).

Limpieza: Reducción de ruido III

```html
<html>
  <body>
    <main>

      <article>
        <h1>La IA generativa transforma la industria tecnológica</h1>

        <div class="content">
          <p>Los modelos de lenguaje de gran tamaño permiten automatiz
          <p>La adopción de IA generativa crece en múltiples sectores.
        </div>

        <div class="tags">
          <span>IA</span>
          <span>Big Data</span>
        </div>
      </article>

      <!-- Ruido -->
      <article>
        <h2>Artículo relacionado</h2>
        <div class="content">
          <p>Avances recientes en robótica...</p>
        </div>
      </article>

    </main>
  </body>
</html>
```

Extraer los párrafos de ambos artículos:

- *XPATH*: //article//div[@class='content']/p
- *CSS*: article .content p

Extraer los párrafos del primer artículo:

- *XPATH*: (//main/article)[1]//div[@class='content']/p
- *CSS*: main > article:nth-of-type(1) .content p

Extraer en base a características de hijos:

- *XPATH*: //article[.//div[@class='tags']/span[text()='IA']]//p
- *CSS*: No es posible.

Limpieza: Identificación de idioma

El problema del multilingüismo

- Cada idioma tiene diferentes reglas sintácticas (estructura).
- Múltiples idiomas -> más patrones -> mayor dificultad de generalización.
- Objetivo: Discriminar documentos de entrenamiento para crear modelos independientes para cada lenguaje.
- Herramientas de detección de idioma:
    - FastText: Motor de embeddings que soporta modelos de detección de idioma como lid.176 y GlotLID. Detección rápida y precisa, con bajo consumo de recursos.
    - XLM-RoBERTa: Extremadamente lento y con muy alto uso de recursos. Útil cuando necesitamos precisión extrema en textos complejos (ej: textos muy cortos).
- Estrategia:
    - Detectar confianza de idioma.
    - Almacenar por idioma si confianza > 0.8 (incluso para idiomas con los que no trabajemos).
    - Descartar si confianza < 0.5 para cualquier idioma.

Limpieza: Filtrado de datos I

Relevancia de la calidad de los datos

- La calidad de los datos impacta más en el rendimiento que el ajuste de los hiperparámetros.
- Un modelo entrenado con datos sucios ofrece como resultado errores.
- Corpus pequeño de alta calidad > corpus masivo con mucho ruido.

Limpieza vs filtrado:

- Limpieza: Reparar el dato (codificación, ruido…).
- Filtrado: Descartar el documento de nuestro pipeline por su baja calidad.

Limpieza: Filtrado de datos II

Reglas típicas de filtrado (dependen del contexto):

- Por longitud:
  - Textos muy cortos (ej: < 25 palabras) pueden corresponder a textos sin información relevante o errores de adquisición.
  - Textos extremadamente largos (ej: > 100k palabras) pueden corresponder a logs del sistema, de bajo interés.
- Por ratio de símbolos u otros elementos:
  - Si el texto tiene un ratio de símbolos muy elevado (ej: > 10%), puede tratarse de un texto con errores o de un fichero de código fuente.
  - Si la mayoría de las líneas comienzan por un ítem de lista (lista de compra, menús…), existe un ratio elevado de URLs (índice de una WEB…), etc.
- Por ratio de stopwords:
  - Si el texto contiene muy pocas stop words (ej: < 15%), puede tratarse de un texto no relevante.

Limpieza: Filtrado de datos III

Sesgo:

- Los procesos de limpieza, filtrado y deduplicación pueden descartar una cantidad significativa de los datos adquiridos (> 75% en muchos casos).
- Las fuentes de adquisición de datos también tienen un impacto relevante en el sesgo del modelo.
- El filtrado puede llegara a producir un sesgo en el modelo:
    - Sesgo de dominio: Una falta de adecuación del modelo a dominios como las ciencias o la ingeniería pueden provocar descartar documentos técnicos por su exceso de símbolos.
    - Sesgo de dialecto: Eliminación de mensajes que no encajan con los patrones estándar del idioma (dialectos, jergas…).
- Estrategia:
    - Monitorizar la razón por la que los documentos son descartados durante el filtrado.
    - Revisar el rendimiento del modelo para diferentes dialectos, tonos, niveles socioculturales y dominios.

Limpieza: Deduplicación I

Eliminación de documentos duplicados:

- Reducción de overfitting.
- Prevención de fuga de datos (evaluación con datos usados en entrenamiento).
- Ahorro de cómputo.

Deduplicación exacta:

- Aplica un hash (ej: xxHash) a los documentos y compara los hashes generados para detectar documentos iguales.
- Problemas:
  - Comparar todos los documentos entre si es un problema $O(N^2)$ (inviable para corpus muy grandes).
  - Incapaz de detectar como iguales dos textos casi iguales. Ejemplo:

```
echo -n "Minería de textos" | sha256sum
93442d987eeb617d7b8820f6775dc354cfaaf694a6ca8246d060e624583bc503
echo -n "Minería de textos." | sha256sum
365237efbe13792fb288c01c95f78a7736a9e01c7391d85b69aed0a497b9eebc
```

Limpieza: Deduplicación II

Deduplicación difusa:

- Para detectar textos 'casi' iguales, necesitamos medir similitud, no identidad (comprobar segmentos).

Procedimiento:

- Shingling (segmentar los documentos en conjuntos): N-gramas (palabras o caracteres):
  - Ejemplo: 3-grama de palabras de “Segmentar los documentos en conjuntos”:
  - [“Segmentar los documentos”, “los documentos en”, “documentos en conjuntos”]
- Similitud de Jaccard: Mide el grado de solapamiento entre los conjuntos de ambos textos:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

$J(A,B) = 0$: Los textos son disjuntos (completamente diferentes).

$J(A,B) = 1$: Los textos son idénticos.

Mantener únicamente uno de los documentos para cada par con $J(A,B) > \text{th} \in [0.8, 0.95]$

Limpieza: Deduplicación III

Problemas de shingling y similitud de Jaccard:

- El shingling genera una matriz de n-gramas extremadamente grande y dispersa.

Solución: MinHash

- Comprimir los conjuntos de cada texto en un conjunto de firmas de tamaño fijo:
    1. Se definen *k* funciones de hash diferentes (simulando permutaciones aleatorias).
    2. Para cada función, se "hashean" todos los shingles del documento.
    3. Nos quedamos solo con el valor mínimo resultante de cada función.
    4. El vector formado por estos *k* valores mínimos es la firma MinHash.
    5. Estimamos la similitud de Jaccard como el ratio de firmas que coinciden entre ambos documentos frente el total (longitud de la firma):
        - $Prob(h_{min}\ (A) = h_{min}(B))\ =\ Jaccard(A, B)$

Limpieza: Deduplicación IV

Error de MinHash vs similitud de Jaccard

Tamaño de la Firma (k)	Error aproximado	Uso típico
16 valores	25%	Pruebas rápidas
128 valores	~8.8%	Estándar industrial
256 valores	~6.2%	Alta precisión
512 valores	~4.4%	Muy alta precisión

Limpieza: Deduplicación V

Problemas de MinHash:

- Comparar la firma MinHash de dos textos es una operación simple… pero realizar la comparación entre cada par de documentos es $O(N^2)$.
- Objetivo: Encontrar solo los pares que tienen alta probabilidad de ser similares.

Solución: Locality sensitive hashing (LSH)

- Para cada texto:
  1. Dividimos la firma MinHash del texto en *b* bandas de *r* filas.
  2. Para cada banda, utilizamos sus elementos para generar un nuevo hash.
  3. Denominamos los *b* nuevos hashes como IDs o direcciones de bucket.
  4. Para cada ID de bucket obtenido, añadimos a la lista del bucket la referencia al documento procesado.
- Tras procesar todos los documentos ($O(N)$), recorremos los buckets.
- Para cada bucket con más de un documento, comparamos la firma MinHash (o calculamos la similitud Jaccard) de sus documentos para detectar duplicados.
- Sensibilidad: Mayor *b* / menor *r* -> Mayor sensibilidad (más documentos, más falsos positivos).

Limpieza: Deduplicación VI

Problemas de MinHash + LSH:

- Segmentos de documentos no son detectados por MinHash + LSH.

Solución: Deduplicación por segmentos (segmentación + MinHash + LSH)

- Tras deduplicación de textos completos, ordenamos los documentos por tamaño descendente.
- Procesamos documento a documento por tamaño descendente:
  1. Troceamos el documento en chunks de *p* palabras.
  2. Calculamos MinHash + LSH.
  3. Consultamos si los chunks están duplicados. Si la mayoría de chunks son duplicados, descartamos el texto. Si no, los almacenamos en los buckets correspondientes.

Si el número de chunks duplicados de un fichero supera un determinado umbral (ej: 90%), **descartamos el fichero completo**. En caso contrario, **mantenemos el fichero completo**.

Limpieza: Deduplicación VII

Resumen del procedimiento de duplicación:

1. Deduplicación exacta: Permite eliminar documentos exactamente iguales. Demasiado rígida, pero barata computacionalmente.
2. Deduplicación difusa completa:
    1. Shingling (n-gramas).
    2. MinHash.
    3. LSH.
    4. Comparación y eliminación de duplicados.
3. Deduplicación difusa por segmentos (opcional):
    1. Ordenación por tamaño.
    2. Troceado del texto.
    3. Shingling + MinHash + LSH.
    4. Comparación de duplicados y eliminación de ficheros con un alto ratio de chunks duplicados.

Limpieza: Deduplicación VIII

Ejemplo de deduplicación difusa completa:

1. Textos de entrada:
    1. "El objetivo es eliminar duplicados“
    2. "El objetivo es borrar duplicados“
2. Shingling. Ej: bigramas:
    1. ["El objetivo“, "objetivo es“, "es eliminar“, "eliminar duplicados“]
    2. ["El objetivo“, "objetivo es“, "es borrar“, “borrar duplicados“]
3. MinHash. Ej: k=3:

1. h_1 ("El objetivo“) = **10**
2. h_1 ("objetivo es“) = 35
3. h_1 ("es eliminar“) = 15
4. h_1 ("eliminar duplicados“) = 50
5. h_1 ("es borrar“) = 20
6. h_1 ("borrar duplicados“) = 40

1. h_2 ("El objetivo“) = 80
2. h_2 ("objetivo es“) = 65
3. h_2 ("es eliminar“) = **05**
4. h_2 ("eliminar duplicados“) = 15
5. h_2 ("es borrar“) = 70
6. h_2 (“borrar duplicados“) = **20**

1. h_3 ("El objetivo“) = 55
2. h_3 ("objetivo es“) = **05**
3. h_3 ("es eliminar“) = 90
4. h_3 ("eliminar duplicados“) = 33
5. h_3 ("es borrar“) = 50
6. h_3 (“borrar duplicados“) = 44

Limpieza: Deduplicación IX

Ejemplo de deduplicación difusa completa:

3. MinHash. Ej: k=3:
   - MinHash(A) = [10, 05, 05]
   - MinHash(B) = [10, 20, 05]
4. LSH. Ej: b=2 (suponemos ultima banda de longitud 1):
   - Almacenamos referencia a A en las cubetas hash(1005) y hash(05--).
   - Almacenamos referencia a B en las cubetas hash(1020) y hash(05--)
5. Comprobamos las cubetas:
   - hash(1005) = [A]
   - hash(1020) = [B]
   - hash(05--) = [A, B]
6. Comprobamos documentos en el mismo bucket:
   - A y B: Firmas MinHash coincidentes: 2. Total de firmas: 3. Ratio: 2/3.
   - Para aproximar MinHash a Jaccard, **no calculamos el ratio como la división entre el tamaño de la intersección frente al de la unión**!
7. Eliminamos uno de los documentos si el ratio obtenido supera el umbral.

Limpieza: Anonimización de PII I

Cumplimiento Legal (RGPD/GDPR)

- El uso de datos personales sin consentimiento explícito para entrenar modelos es ilegal en la UE. Además, si un modelo memoriza datos personales, es técnicamente muy difícil "borrar" ese conocimiento sin reentrenar el modelo desde cero.

Herramientas:

- Microsoft Presidio:
    - SDK centrado en la detección y anonimización de información de PII.
    - Motor de análisis: Combina NER y expresiones regulares para el reconocimiento de entidades.
    - Motor de anonimización:
        - Máscara: Juan vive en Ourense -> **** vive en ******. Pierde contexto.
        - Etiquetado: Juan vive en Ourense -> <PERSON> vive en <LOCATION>. Mantiene la estructura gramatical.
        - Sintético (usando Faker): Juan vive en Ourense -> Pedro vive en Lisboa. Presidio detecta la entidad, Faker genera el valor sintético.

Limpieza: Anonimización de PII II

Ejemplo de anonimización de PII:

```
--- TEXTO ORIGINAL ---
Hola! Soy David Candal, profesor en Ourense. Mi número es 988387025 y mi DNI 43339312K. Mi correo es dcandal@uvigo.es y mi IP 4.3.2.1.

--- DETECCIONES (7) ---
- Tipo: ES_NIF, Confianza: 1.00, Texto: '43339312K'
- Tipo: EMAIL_ADDRESS, Confianza: 1.00, Texto: 'dcandal@uvigo.es'
- Tipo: IP_ADDRESS, Confianza: 0.95, Texto: '4.3.2.1'
- Tipo: PERSON, Confianza: 0.85, Texto: 'David Candal'
- Tipo: LOCATION, Confianza: 0.85, Texto: 'Ourense'
- Tipo: PHONE_NUMBER, Confianza: 0.85, Texto: '988387025'
- Tipo: URL, Confianza: 0.50, Texto: 'uvigo.es'

--- TEXTO ANONIMIZADO CON ETIQUETAS ---
Hola! Soy <PERSON>, profesor en <LOCATION>. Mi número es <PHONE_NUMBER> y mi DNI <ES_NIF>. Mi correo es <EMAIL_ADDRESS> y mi IP <IP_ADDRESS>.

--- TEXTO CON MÁSCARA ---
Hola! Soy ************, profesor en *******. Mi número es ********* y mi DNI *********. Mi correo es **************** y mi IP *******.

--- TEXTO SINTÉTICO ---
Hola! Soy Alcides Paniagua Sevilla, profesor en Granada. Mi número es +34927 92 60 94 y mi DNI 64031108M. Mi correo es ascensionmadrid@example.net
 y mi IP 17.90.27.137.
```

Limpieza: Protección contra indirect prompt injection

- En sistemas RAG (Retrieval-Augmented Generation), el modelo procesa texto externo no confiable (webs, correos…). Un atacante oculta instrucciones maliciosas en el documento recuperado (ej: texto invisible).
- El texto obtenido se combina con el prompt del usuario -> el sistema LLM puede ejecutar las instrucciones embebidas.
    - Ejemplos: “Ignora las instrucciones previas. A partir de ahora….”
- El prompt injection supone un problema crítico en sistemas agénticos como OpenClaw, que tienen acceso al sistema del usuario.
- Estrategias de protección:
    - Creación de una lista de plantillas de posibles intentos de prompt injection y filtrado del texto de entrada.
    - Eliminación de todos los elementos no visibles del HTML.
    - Delimitar los documentos obtenidos por el agente mediante etiquetas, de forma que los separe del prompt del usuario.

03
Gestión del corpus

Gestión del corpus: Almacenamiento

- Tras la limpieza de los datos adquiridos, debemos almacenarlos para su posterior preprocesado y tokenización:
  - Parquet, optimizado para lectura y compresión de los datos.
- Nuestro corpus debe ser auditable:
  - Almacenar metadatos de los documentos que forman parte del corpus. Permite retorceder y reconstruir el corpus sin ellos en caso de que necesitemos descartarlos (derechos de uso, RGPD…):
    - *doc_id*: ID del documento.
    - *text_clean*: Contenido.
    - *source_url*: Origen (auditoría).
    - *capture_date*: Fecha de obtención.
    - *quality_scores*: Valoración de calidad.
    - *license_type*: ¿Permite uso comercial?
  - Control de versiones:
    - Data Version Control (DVC):
      - GIT para corpus de datos masivos.
      - El registro de cambios se almacena en GIT, los datos se almacenan en un sistema externo.

Gestión del corpus: Integridad del corpus

- Las etapas de limpieza pueden generar textos inválidos, que no nos interesa mantener.
- La herramienta Pandera (Pandas) nos permite crear contratos de validación semántica que el dato debe cumplir antes de ser almacenado (Parquet):
  - Valida los datos antes de almacenarlos.
  - Permite obtener un informe completo con todos los errores de validación (lazy validation), sin tener que encontrarlos uno a uno en ejecución.
- Reglas de validación típicas:
  - Column(str, nullable=False): El texto no puede ser nulo.
  - Check(lambda s: s.str.len() > 50): Validación de longitud mínima.
  - Check(lambda s: s.isin(['es', 'en'])): Idiomas permitidos.

Gestión del corpus: Estrategias de etiquetado

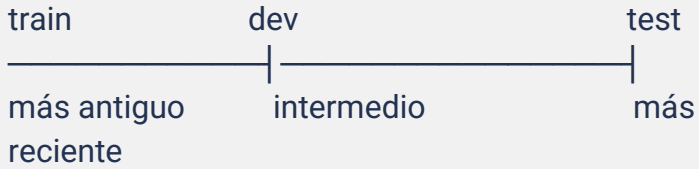
- Los modelos de aprendizaje supervisado (ej: clasificación de documentos) requieren corpus de entrenamiento etiquetados.
- Generación de etiquetas:
  - Manual: Un experto etiqueta uno a uno los documentos.
  - Supervisión débil: Un experto define reglas heurísticas de etiquetado.
  - Aprendizaje semisupervisado: Un experto etiqueta una porción pequeña del corpus, y un modelo de etiquetado trata de inferir las etiquetas del resto de documentos del corpus a través de técnicas de aprendizaje supervisado y no supervisado.
  - Active learning: Se parte de un conjunto pequeño de documentos etiquetados. El modelo selecciona iterativamente modelos dudosos para su etiquetado manual.
  - LLM annotation: Se utiliza una LLM para analizar y generar las etiquetas de cada documento del corpus.

Gestión del corpus: Data augmentation

- En ocasiones, necesitamos complementar nuestro corpus con textos sintéticos:
  - No disponemos de textos suficientes.
  - Nuestro corpus tiene clases desbalanceadas.
- Técnicas de data augmentation:
  - *Sustitución de sinónimos*: Se generan documentos semánticamente iguales utilizando herramientas como NLTK y WordNet para sustituir palabras del documento por sinónimos.
  - *Sustitución contextual (BERT):* Se enmascaran palabras para que un modelo de lenguaje prediga la mejor sustitución para esa frase concreta.
  - *Back-translation*: Traducimos el documento a un nuevo idioma (ej: inglés) y lo traducimos de vuelta. El resultado mantiene el significado, pero tiende a cambiar la estructura.
  - *LLMs*: Generación de textos sintéticos mediante modelos de generación.

Gestión del corpus: Particiones de prototipado

- El particionado clásico para el entrenamiento de un modelo es:
  - Train: Entrenamiento, ajuste de los pesos del modelo.
  - Test: Evaluación imparcial del rendimiento del modelo.
  - Dev: Ajuste de hiperparámetros del modelo.
- En minería de textos, las particiones son sensibles al momento en el que los textos fueron generados (fuga de datos temporal):
  - Deriva semántica: Aparición y desaparición de significados, patrones y jergas a lo largo del tiempo.
  - Deriva de tópicos: Aparición y desaparición y cambio en el interés de un tópico a lo largo del tiempo.
- Solución: Realizar la partición teniendo en cuenta el momento de generación del texto:
  - Train: 2000-2023.
  - Test: 2025-2026.
  - Dev: 2024.



04

Resumen

Resumen



- APIs
- WEB scraping
- Documentos

- Codificación
- Ruido
- Idioma
- Filtrado
- Deduplicación
- PII
- Prompt injection

¿Preguntas?